

Q1:

- **Size** - A total number of elements currently stored in an array.
- **Capacity** – The maximum number of elements that can be held in the currently allocated memory block.

Q2:

- I. If there is a space in memory after the end of the array, then it can simply extend it which copying the data. It's a $O(1)$.

Before:

A	B	C	D	Free Space
---	---	---	---	------------

After:

A	B	C	D	E	Free Space
---	---	---	---	---	------------

- II. If the memory after the end of the array is occupied by another variable, then the in-place extension is not possible. So, it must allocate a new block of the updated array size and copy all the elements to that and free the old block. Its $O(n)$ because we need copy the data every time a new element is added.

Before:

old:	A	B	C	D	X(different variable)
------	---	---	---	---	-----------------------

After:

New:	A	B	C	D	E
------	---	---	---	---	---

Q3:

Real-world array implementations (such as C++ `std::vector`, Java `ArrayList`, and Python `list`) use dynamic resizing strategies to achieve an **amortized constant time cost** for insertions, even though individual expansion operations are. By spreading the high cost of reallocating memory and copying elements over many inexpensive "cheap" insertions, the overall performance is optimized.